

Contents

1	introduction	2
2	Physics-Informed Neural Networks	4
2.1	PINN Formulation for the Schrödinger Equation	6
3	Results and Discussion	9
4	Conclusion	17

1 introduction

Physics-informed neural computations have their theoretical underpinnings in the universal approximation theorem formulated by Hornik et al. [5]. The paper demonstrated that multilayer feedforward neural networks can approximate classes of continuous and measurable functions with arbitrary accuracy. This theoretical result is of great importance because it gave rise to the idea of representing unknown physical fields using the neural networks and, consequently, the solution of differential equations.

In the 1990s, efforts to combine neural networks with physical knowledge and differential equation modelling began. An early example was proposed by Psychogios and Ungar [11] when they presented the hybrid neural network-first-principles approach for process modeling. Here, the neural network served as a tool to approximate the unknown process parameters, whereas the known part of the system was modeled through physical relations like mass and energy balances. Even though the neural network was not used to approximate the solution of the differential equation in this case, this was one of the first attempts to incorporate prior physical knowledge into the neural network models.

Building on the universal approximation theorem proposed by Hornik et al. [5], the unknown solution of the PDE was approximated by a multi-layered feed-forward neural network as suggested by Dissanayake and Phan-Thien [3]. The parameters of the network were adjusted such that the objective function made up of the residual from the governing-equation and boundary conditions at certain collocation points was minimized. Given the similarity of the formulation of the objective function with those of current PINNs, the method suggested by Dissanayake and Phan-Thien might be regarded as an important precursor to modern physics-informed neural networks, even though physics-informed neural networks had not been invented yet.

Later in the decade, Lagaris et al. [9] proposed a framework for solving initial/boundary value problems defined by both ODEs and PDEs. Their framework included constructing the trial solutions through a known part satisfying the initial/boundary conditions and a variable part obtained by a neural network.

During the 2000s and early 2010s, improvements in computational power and numerical optimization led to the creation of highly developed techniques of solving differential equations using neural-network methods. As reviewed by Kumar and Yadav [8], a wide range of methods were proposed based on multilayer perceptron, radial-basis-function, hybrid, and collocation methods of modeling ODEs and PDEs. The emergence of deep learning further increased the capacity of neural networks to perform complex computations, allowing researchers to train models consisting of multiple processing layers and therefore learn more complicated functions [10]. Moreover, the rise of software platforms, such as TensorFlow and Keras made building and differentiating deep neural networks easier, faster and more convenient [1, 6]. The first one provides a computation graph and automatic differentiation, making neural networks scalable across CPUs and GPUs, while the second allows defining and training the network architecture in a straightforward way.

In parallel with the above progress, researchers also investigated the use of deep learning for structured scientific problems. For instance, Kondor and Trivedi [7] presented a theory on how symmetry/equivariance can be considered within neural network frameworks, while Vlachas et al. [15] showed that LSTM networks can predict chaotic high-dimensional dynamical systems based solely on the input data. These works were not directly related to physics-informed neural networks, but they made significant contribution to the progress in deep-learning techniques for modelling physical and dynamical systems. Building upon earlier neural-network approaches for differential equations,

advances in deep learning and automatic differentiation, and the growing integration of physical knowledge into machine learning, Raissi et al. [12] developed the modern PINN formulation for nonlinear PDEs.

Following the work of Raissi, Haghighat and Juanes [4] introduced SciANN, a TensorFlow- and Keras-based library developed for scientific computing and physics-informed deep learning. The applications of PINNs have expanded rapidly across a wide range of scientific and engineering disciplines. Researchers have successfully applied PINNs to nonlinear fluid dynamics, inverse parameter identification, computational finance, additive manufacturing, and other data-driven scientific computing problems [18, 17, 14, 13]. These studies demonstrate the versatility of PINNs in solving both forward and inverse PDE problems while effectively incorporating physical constraints into the learning process.

In this case, FDM and PINNs are used to find solutions for the one-dimensional time-dependent Schrödinger equation. The finite difference method is a traditional numerical approach where the differential operator in the equation is approximated at discrete points in a numerical grid. The physics-informed neural networks, however, use the loss function, where the PDE is part of the loss function and the conditions are added separately, while the neural network attempts to minimize this loss function. FDM uses mesh discretization, whereas PINN can produce continuous solutions using automatic differentiation without a mesh. The results from both methods will be compared to the analytical solution in this paper.

2 Physics-Informed Neural Networks

The conventional neural network is trained with the help of data only. If we provide some data samples to the network, the network tries to minimize the error between the predicted output values and the observed values. Though this methodology has produced very good results in many different scenarios, it needs plenty of data and does not ensure that the solution that is predicted by the model complies with the physical laws.

On the other hand, Physics-Informed Neural Networks (PINNs) combine the concept of a neural network with the physical laws governing the system. Unlike a standard neural network, which is trained using data only, a PINN is constrained by physical laws represented by a Partial Differential Equation (PDE). Here, the neural network is responsible for approximating the unknown solution ($u(t, x)$), whereas automatic differentiation is used to calculate the necessary derivatives in the PDE. For a general PDE of the form

$$u_t + \mathcal{N}[u] = 0 \quad (1)$$

the neural network predicts the solution ($u(t, x)$). Using automatic differentiation, the derivatives (u_t), (u_x), (u_{xx}), and higher-order terms can be computed exactly with respect to the network inputs. These derivatives are then substituted into the governing equation to form a residual function

$$f(t, x) = u_t + \mathcal{N}[u]. \quad (2)$$

In case the neural network adheres exactly to the physical law, then the residual ($f(t, x)$) is supposed to take the value of zero throughout the whole domain. Hence, there will be two components in

the training process of a PINN. One is related to the data error while the other component relates to physics, which serves as an additional constraint.

A PINN may therefore be regarded as a neural network that makes use of both observation data as well as the principles of physics. The incorporation of such physical knowledge provides additional information for the network, which works as a regularization technique.

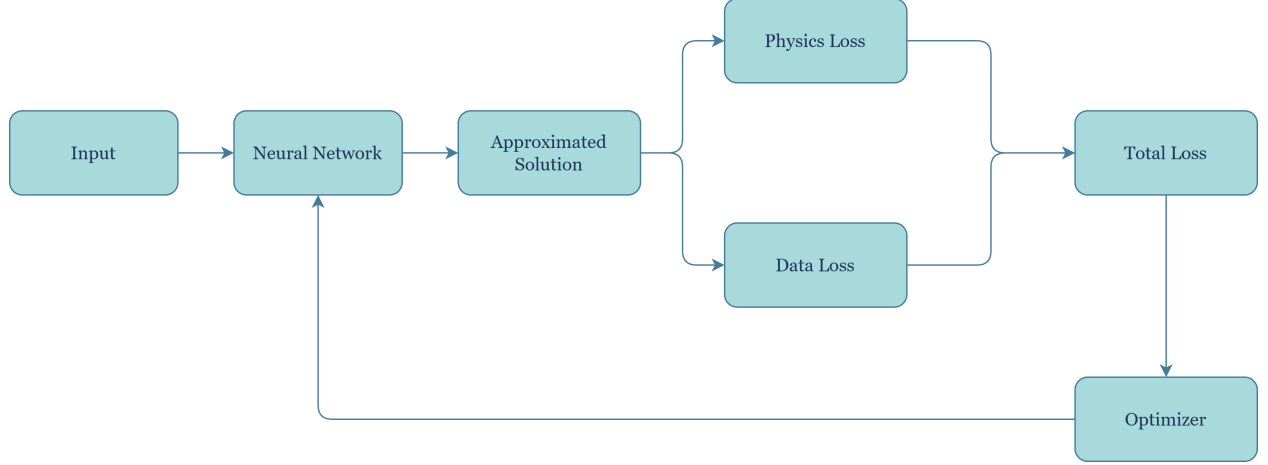


Figure 1: PINN Framework

The architecture of a Physics-Informed Neural Network is typically implemented based on a fully connected feed-forward neural network, though in many cases they try many other architecture for NN. In this structure, the input layer receives the independent variables of the problem. For a time-dependent one-dimensional PDE, these inputs are usually the spatial coordinate (x) and the time variable (t). Therefore, each input point can be shown as (x, t) . The role of the neural network is mapping this input point to the corresponding approximation of the unknown solution. In other words, the network represents the function

$$u_{\theta}(x, t) \approx u(x, t) \quad (3)$$

where (θ) denotes all trainable parameters of the neural network, including weights and biases. In the traditional neural network, the output of the architecture is checked only with available data. However, the output from the architecture in a PINN is further taken to the physics-based part of the algorithm. This means that after the network predicts $u_{\theta}(x, t)$, automatic differentiation is applied to compute the derivatives required in the PDE, such as

$$\frac{\partial u_{\theta}}{\partial t}, \frac{\partial u_{\theta}}{\partial x}, \frac{\partial^2 u_{\theta}}{\partial x^2}. \quad (4)$$

To evaluate how well the governing equation is satisfied, PINNs introduce a set of collocation points inside the computational domain. These points are sampled throughout the domain and are not necessarily associated with any observed data. Let the collocation points be denoted by

$(x_f^i, t_f^i)_{i=1}^{N_f}$ where (N_f) represents the number of collocation points. At each of these points, the residual $(f_{\theta}(x, t))$ is evaluated. If the residual is large at a particular location, it indicates that the neural

network prediction violates the governing equation at that point. Consequently, the network parameters are adjusted to reduce this error. One important advantage of this approach is that PINNs are mesh-free methods and therefore do not require the computational mesh used by traditional numerical techniques such as the Finite Element Method or Finite Difference Method[12] [2].

The learning process of a PINN is controlled through a loss function that combines information from both the available data and the governing physics. In the most general case, the total loss function consists of four components:

$$\mathcal{L}_{data} + \mathcal{L}_{physics} + \mathcal{L}_{boundary} + \mathcal{L}_{initial}$$

The data loss measures the difference between the neural network predictions and the available observations. The physics loss evaluates how well the governing differential equation is satisfied at the collocation points. The boundary loss enforces the prescribed boundary conditions, while the initial loss ensures that the initial conditions are satisfied. Together, these terms guide the network toward a physically meaningful solution. More specifically, the physics loss is often defined as the mean squared value of the PDE residual evaluated at all collocation points:

$$\frac{1}{N_f} \sum_{i=1}^{N_f} |f_{\theta}(x_f^i, t_f^i)|^2 \quad (5)$$

Minimizing this term forces the neural network to satisfy the governing equation throughout the computational domain. Similarly, if observational data are available, the data loss can be written as

$$\frac{1}{N_u} \sum_{i=1}^{N_u} |u_{\theta}(x_u^i, t_u^i) - u^i|^2, \quad (6)$$

where (u^i) denotes the observed value of the solution at the point $((x_u^i, t_u^i))$.

After the total loss function is constructed, an optimization algorithm such as Adam or L-BFGS is used to update the trainable parameters (θ) . During each training iteration, the gradients of the loss function with respect to the network parameters are computed using backpropagation. The optimization process continues until the total loss reaches a sufficiently small value, indicating that the neural network simultaneously fits the available data and satisfies the governing physical laws. The Schrödinger equation is the fundamental equation of quantum mechanics and describes the evolution of a quantum system through its wave function. In its time-dependent form, the equation is written as

$$i\hbar \frac{\partial \psi}{\partial t} = \hat{H} \psi, \quad (7)$$

where $\psi(x, t)$ denotes the wave function, $\hbar$ is the reduced Planck constant, and $\hat{H}$ is the Hamiltonian operator. The solution of the Schrödinger equation contains complete information about the physical system and can be used to determine quantities such as probability distributions and energy levels.

The Schrödinger equation plays a central role in many scientific fields, including quantum mechanics, quantum chemistry, atomic physics, condensed matter physics, and quantum computing. Because analytical solutions are available only for a limited number of simple systems, numerical methods are often required to approximate its solutions.

In recent years, Physics-Informed Neural Networks (PINNs) have emerged as a promising alternative to traditional numerical approaches. By embedding the governing differential equation directly into the loss function, PINNs can learn solutions that satisfy both observational data and the underlying physical laws. This capability makes PINNs particularly attractive for solving Schrödinger-type equations, where accurate approximations of wave functions are required [16].

2.1 PINN Formulation for the Schrödinger Equation

In this project, we consider the one-dimensional time-dependent Schrödinger equation with a harmonic potential

$$i\frac{\partial\psi}{\partial t} = -\frac{1}{2}\frac{\partial^2\psi}{\partial x^2} + \frac{1}{2}x^2\psi, \quad (8)$$

where $\psi(x, t)$ denotes the complex-valued wave function. The term

$$-\frac{1}{2}\frac{\partial^2\psi}{\partial x^2} \quad (9)$$

represents the kinetic energy contribution, while

$$\frac{1}{2}x^2\psi \quad (10)$$

corresponds to the harmonic oscillator potential. This equation is one of the most important models in quantum mechanics and is frequently used as a benchmark problem for numerical and machine learning methods.

To solve this problem using PINNs, the neural network receives the spatial coordinate x and time t as inputs and approximates the wave function $\psi(x, t)$. Automatic differentiation is then employed to compute the derivatives required by the governing equation, and the residual of the Schrödinger equation is incorporated into the loss function. The network parameters are optimized such that the predicted solution satisfies both the differential equation and the prescribed initial and boundary conditions.

To obtain a unique solution of the Schrödinger equation, appropriate initial and boundary conditions must be specified. In this work, the spatial domain is defined as

$$x \in [-5, 5] \quad (11)$$

while the temporal domain is chosen as

$$t \in [0, 1] \quad (12)$$

The initial condition is selected as a Gaussian wave packet,

$$\psi(x, 0) = e^{-x^2} \quad (13)$$

which is a commonly used initial state in quantum mechanics due to its smoothness and localized nature. Since the wave function is represented in terms of its real and imaginary components,

$$\psi(x, t) = u(x, t) + iv(x, t) \quad (14)$$

the initial condition can be written as

$$u(x, 0) = e^{-x^2} \quad (15)$$

$$v(x, 0) = 0 \quad (16)$$

These conditions are enforced during training through the initial-condition loss term, which penalizes discrepancies between the neural network predictions and the prescribed initial values. Furthermore, homogeneous Dirichlet boundary conditions are imposed at the boundaries of the spatial domain. Specifically,

$$u(-5, t) = u(5, t) = 0 \quad (17)$$

and

$$v(-5, t) = v(5, t) = 0 \quad (18)$$

These boundary conditions ensure that the wave function remains negligible at the edges of the computational domain, which is consistent with the localized nature of the Gaussian initial state. Within the PINN framework, both the initial and boundary conditions are incorporated directly into the loss function. During training, the neural network is required not only to satisfy the governing Schrödinger equation but also to reproduce the prescribed initial and boundary conditions. This enables the network to learn a physically consistent approximation of the wave function throughout the spatio-temporal domain.

To solve the time-dependent

Schrödinger equation using Physics-Informed Neural Networks (PINNs), the complex-valued wave function is first decomposed into its real and imaginary components. Thus, the solution is represented as

$$\psi(x, t) = u(x, t) + i v(x, t) \quad (19)$$

where $u(x, t)$ and $v(x, t)$ denote the real and imaginary parts of the wave function, respectively.

In the PINN framework, a fully connected neural network is employed to approximate these two functions simultaneously. The inputs of the network are the spatial coordinate x and the time variable t , while the outputs correspond to the approximations $u_\theta(x, t)$ and $v_\theta(x, t)$, where θ denotes the trainable parameters of the neural network.

The governing equation considered in this work is

$$i \frac{\partial \psi}{\partial t} = -\frac{1}{2} \frac{\partial^2 \psi}{\partial x^2} + \frac{1}{2} x^2 \psi \quad (20)$$

Substituting

$$\psi = u + i v \quad (21)$$

into the Schrödinger equation and separating the real and imaginary parts yields the coupled system

$$u_t = -\frac{1}{2} v_{xx} + V(x) v \quad (22)$$

$$v_t = \frac{1}{2}u_{xx} - V(x)u \quad (23)$$

where

$$V(x) = \frac{1}{2}x^2 \quad (24)$$

is the harmonic potential.

Using automatic differentiation, the neural network computes the required derivatives u_t , v_t , u_{xx} , and v_{xx} . These derivatives are then used to construct the physics residuals

$$f_u = u_t + \frac{1}{2}v_{xx} - V(x)v \quad (25)$$

$$f_v = v_t - \frac{1}{2}u_{xx} + V(x)u \quad (26)$$

If the neural network exactly satisfies the Schrödinger equation, both residual functions vanish throughout the computational domain. Therefore, the objective of the PINN is to minimize these residuals while simultaneously satisfying the prescribed initial and boundary conditions.

To enforce the governing equation, a set of collocation points is sampled inside the spatio-temporal domain. At these points, the residuals f_u and f_v are evaluated and incorporated into the loss function. The resulting physics loss is given by

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_f} \sum_{i=1}^{N_f} (|f_u(x_i, t_i)|^2 + |f_v(x_i, t_i)|^2) \quad (27)$$

where N_f denotes the number of collocation points.

The total loss function combines the physics loss with the losses associated with the initial conditions, boundary conditions, and available reference data. Therefore,

$$\mathcal{L} = \lambda_{\text{PDE}}\mathcal{L}_{\text{PDE}} + \lambda_{\text{IC}}\mathcal{L}_{\text{IC}} + \lambda_{\text{BC}}\mathcal{L}_{\text{BC}} + \lambda_{\text{Data}}\mathcal{L}_{\text{Data}} \quad (28)$$

where the coefficients λ_{PDE} , λ_{IC} , λ_{BC} , and λ_{Data} control the contribution of each component during training.

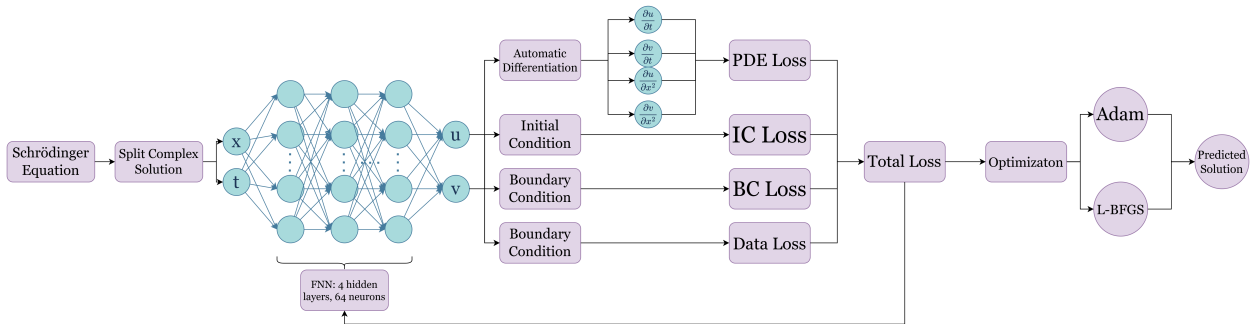


Figure 2: Diagram of solving Schrodinger with PINN

3 Results and Discussion

The performance of the proposed physics-informed neural network was evaluated from four complementary perspectives: training convergence, agreement with the finite-difference reference solution, satisfaction of the governing differential equation, and conservation of the wave-function norm. The results demonstrate that the trained network accurately reproduced the spatio-temporal evolution of the wave function while maintaining small equation residuals and satisfactory physical consistency throughout the computational domain.

The optimization process produced a substantial reduction in all components of the loss function. During the first training stage, the Adam optimizer rapidly moved the network parameters toward a suitable approximation of the solution. The subsequent L-BFGS stage further refined the parameters and reduced the remaining error. This two-stage strategy was particularly effective because Adam provided a stable initial approximation, while L-BFGS improved the final satisfaction of the physical and observational constraints.

The total loss was defined as the sum of the PDE, boundary-condition, initial-condition, and reference-data contributions:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{PDE}} + \mathcal{L}_{\text{BC}} + \mathcal{L}_{\text{IC}} + \mathcal{L}_{\text{data}}. \quad (29)$$

All four grouped loss components decreased by several orders of magnitude during training. The boundary-condition loss reached the smallest final value, indicating that the homogeneous boundary conditions were satisfied with high accuracy. The PDE loss remained the largest component at the end of optimization because the governing equation had to be enforced throughout the entire space-time domain rather than only along the boundaries or initial line. The total training and testing losses followed nearly identical convergence patterns, indicating consistent performance across the sampled training and testing points. This agreement, however, should be interpreted as consistency on the sampled sets rather than definitive proof that overfitting is impossible.

The complete convergence history, including the transition from Adam to L-BFGS, is illustrated in Figure 3.

At the end of training, the total loss reached

$$\mathcal{L}_{\text{total}} = 2.4256 \times 10^{-6}. \quad (30)$$

The combined PDE loss was 1.8756×10^{-6} , accounting for approximately 77.3% of the remaining total loss. By comparison, the boundary, initial-condition, and reference-data losses were 3.0547×10^{-8} , 2.3427×10^{-7} , and 2.8515×10^{-7} , respectively. These values indicate that the network satisfied the auxiliary constraints more accurately than the differential equation itself, which is expected because the PDE residual must be minimized over the full interior of the computational domain.

The real and imaginary PDE losses were similar, suggesting balanced enforcement of the two coupled equations. A greater difference was observed between the initial-condition losses of the two components, with the imaginary-component loss being larger than the real-component loss. Nevertheless, both values remained small and did not prevent the network from achieving high global accuracy. The final individual and grouped loss values are summarized in Table 1.

The accuracy of the trained network was further evaluated by comparing its predicted probability density with the reference solution generated using the finite-difference method. The probability density was calculated from the real and imaginary components as

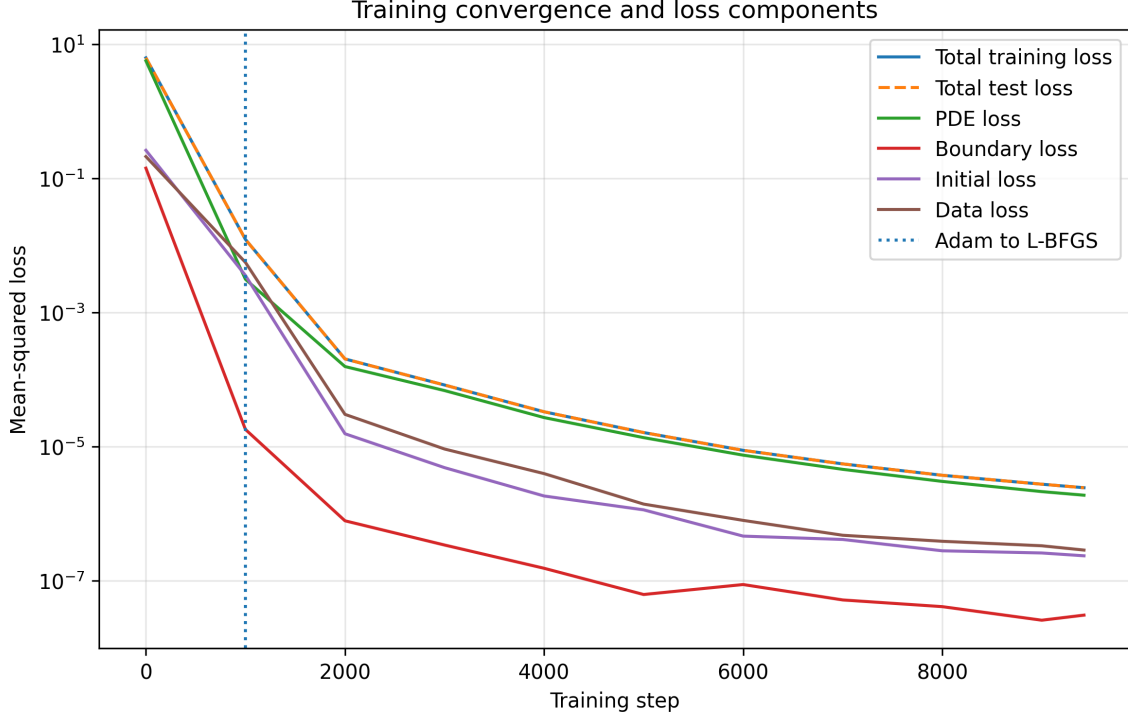


Figure 3: Convergence history of the total training and testing losses and the individual PINN loss components. The vertical dotted line indicates the transition from Adam to L-BFGS.

Table 1: Final individual and grouped PINN loss components.

Loss component	Final value
Real PDE residual loss	9.4813×10^{-7}
Imaginary PDE residual loss	9.2750×10^{-7}
Boundary loss for u	1.7885×10^{-8}
Boundary loss for v	1.2662×10^{-8}
Initial-condition loss for u	2.9938×10^{-8}
Initial-condition loss for v	2.0433×10^{-7}
Reference-data loss for u	1.4485×10^{-7}
Reference-data loss for v	1.4029×10^{-7}
Combined PDE loss	1.8756×10^{-6}
Combined boundary loss	3.0547×10^{-8}
Combined initial-condition loss	2.3427×10^{-7}
Combined reference-data loss	2.8515×10^{-7}
Total training loss	2.4256×10^{-6}

$$\rho(x, t) = |\psi(x, t)|^2 = u(x, t)^2 + v(x, t)^2. \quad (31)$$

Close agreement was obtained between the PINN and FDM probability-density fields throughout the entire computational domain. Both methods produced the same overall spatio-temporal structure, including the central concentration of the wave packet and its gradual broadening over the simulated time interval. No visible displacement or distortion of the principal density structure was observed in the PINN prediction.

The pointwise error was calculated using

$$E_\rho(x, t) = |\rho_{\text{PINN}}(x, t) - \rho_{\text{FDM}}(x, t)|. \quad (32)$$

The largest errors were concentrated near the central region, where the probability density was greatest and changed most rapidly in space and time. Much smaller errors occurred toward the boundaries because the wave-function magnitude approached zero in those regions. This spatial distribution is reasonable because reproducing the height, width, and temporal evolution of the central wave packet is more difficult than approximating nearly zero values far from its centre.

The reference field, PINN prediction, and corresponding absolute-error distribution are compared in Figure 4. An identical colour scale was used for the reference and predicted density fields so that their visual agreement could be assessed directly.

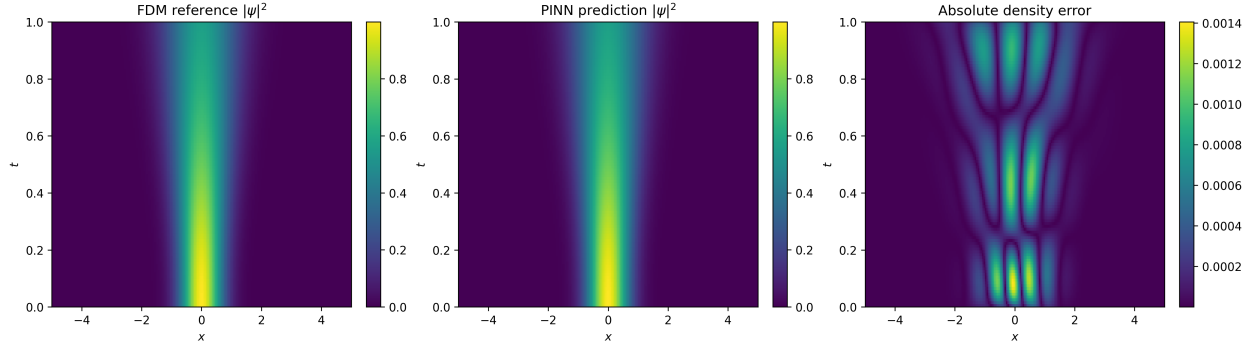


Figure 4: Comparison of the FDM reference probability density, the PINN prediction, and the pointwise absolute density error throughout the space-time domain.

The global relative L_2 error was calculated as

$$\varepsilon_q = \frac{\|q_{\text{PINN}} - q_{\text{FDM}}\|_2}{\|q_{\text{FDM}}\|_2}, \quad (33)$$

where q represents u , v , ψ , or ρ . The resulting relative error of the complete complex wave function was

$$\varepsilon_\psi = 1.5462 \times 10^{-3}, \quad (34)$$

corresponding to approximately 0.155%. The relative error of the probability density was even smaller:

$$\varepsilon_\rho = 9.7108 \times 10^{-4}, \quad (35)$$

which corresponds to approximately 0.097%. Therefore, the global density error remained below one tenth of one percent.

The real component had a relative error of 1.1709×10^{-3} , while the imaginary component had a larger relative error of 3.2844×10^{-3} . This difference does not necessarily indicate a substantial weakness in the prediction of v , because a component-wise relative error also depends on the norm of the corresponding reference component. The error of the complete complex wave function is consequently a more representative measure of the overall solution quality.

The mean absolute density error and root-mean-square density error were 1.2451×10^{-4} and 2.5485×10^{-4} , respectively. The maximum pointwise density error was 1.4054×10^{-3} . The fact that the mean and RMS errors were considerably smaller than the maximum error indicates that the largest discrepancies were localized rather than distributed throughout the domain. The complete accuracy measures are reported in Table 2.

Table 2: Quantitative errors between the PINN and FDM solutions.

Error measure	Value
Relative L_2 error of u	1.1709×10^{-3}
Relative L_2 error of v	3.2844×10^{-3}
Relative L_2 error of ψ	1.5462×10^{-3}
Relative L_2 error of $ \psi ^2$	9.7108×10^{-4}
Density mean absolute error	1.2451×10^{-4}
Density root-mean-square error	2.5485×10^{-4}
Maximum absolute density error	1.4054×10^{-3}

The full-field comparison was complemented by examining the probability-density profiles at representative time instants. This comparison provides a more detailed view of whether the network reproduced the local peak, width, symmetry, and spatial decay of the wave packet.

At $t = 0$, the predicted density closely matched the prescribed initial profile. As time progressed, the central peak gradually decreased while the distribution became wider. This behaviour indicates a redistribution of the wave-function density over a broader spatial region. The reduction in peak height does not represent a loss of norm because the area under the density profile remained approximately constant.

The PINN and FDM curves overlapped closely at

$$t = 0, \quad 0.25, \quad 0.50, \quad 0.75, \quad 1.00.$$

The agreement was maintained not only near the maximum density but also along the tails of the distribution. Consequently, the network accurately reproduced both the amplitude and the spatial broadening of the solution over the complete temporal interval. These selected-time comparisons are presented in Figure 5.

The near-complete overlap of the curves is consistent with the small global density errors reported previously. More importantly, the line profiles confirm that the strong agreement observed in the heatmaps was not caused by colour-scale smoothing. The predicted and reference solutions remained close when compared directly as one-dimensional functions of the spatial coordinate.

Satisfaction of the governing equation was investigated by evaluating the two coupled residuals produced by the real-valued formulation of the Schrödinger equation:

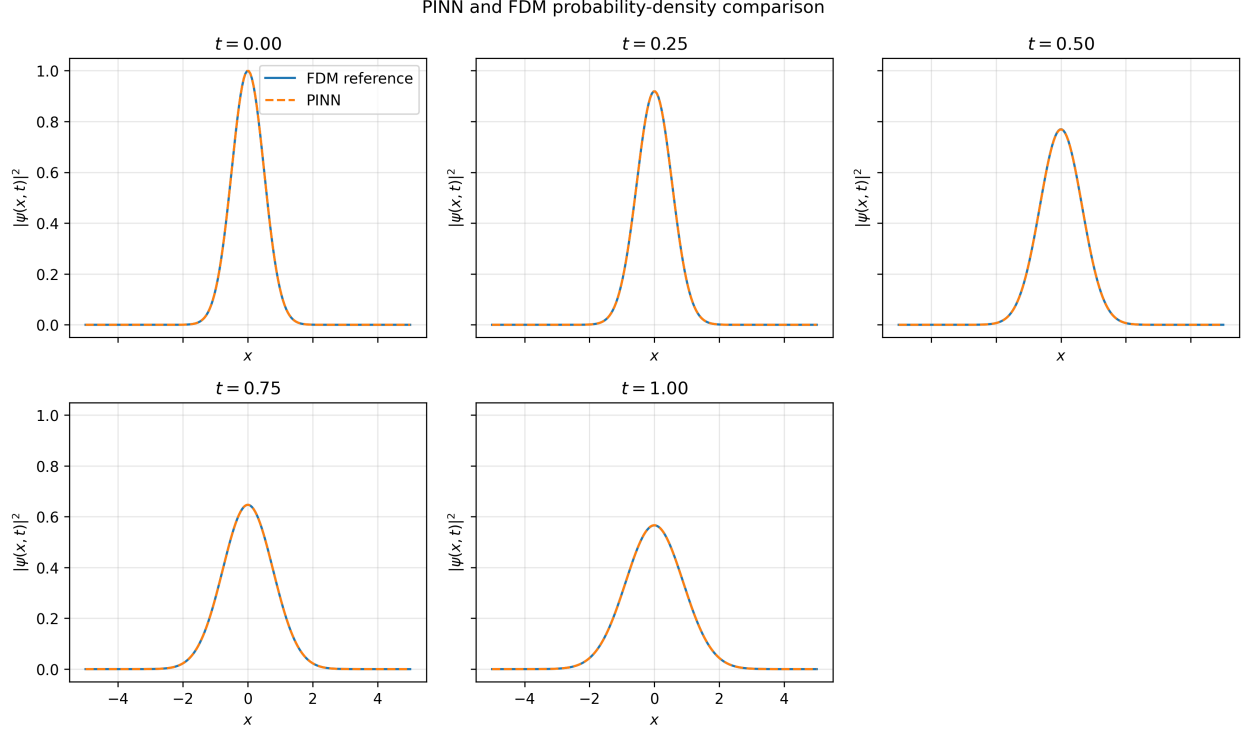


Figure 5: Comparison of the PINN and FDM probability-density profiles at selected time instants.

$$f_u(x, t) = u_t + \frac{1}{2}v_{xx} - V(x)v, \quad (36)$$

$$f_v(x, t) = v_t - \frac{1}{2}u_{xx} + V(x)u. \quad (37)$$

For an exact solution, both residuals would be zero throughout the computational domain. The trained PINN produced residuals that remained small over most of the space-time region. Their magnitudes generally occurred around 10^{-3} or below, although several localized regions reached values of several 10^{-3} .

The residual distributions contained curved low-value bands where the signed residuals passed through zero. These structures do not represent discontinuities in the wave function. They result from plotting the absolute residual magnitude, which becomes very small wherever the signed residual changes sign.

A logarithmic colour scale was used because the residual values covered several orders of magnitude. The resulting distributions of $|f_u|$ and $|f_v|$ are shown in Figure 6.

The mean absolute residuals were 7.5442×10^{-4} and 7.5263×10^{-4} for the real and imaginary equations, respectively. Their RMS values were approximately 1.0×10^{-3} , whereas the maximum residuals were 7.4639×10^{-3} and 6.7442×10^{-3} .

These values are consistent with the final PDE loss. Since the PDE loss is based on squared residuals, an RMS residual of approximately 10^{-3} produces a squared contribution of approximately

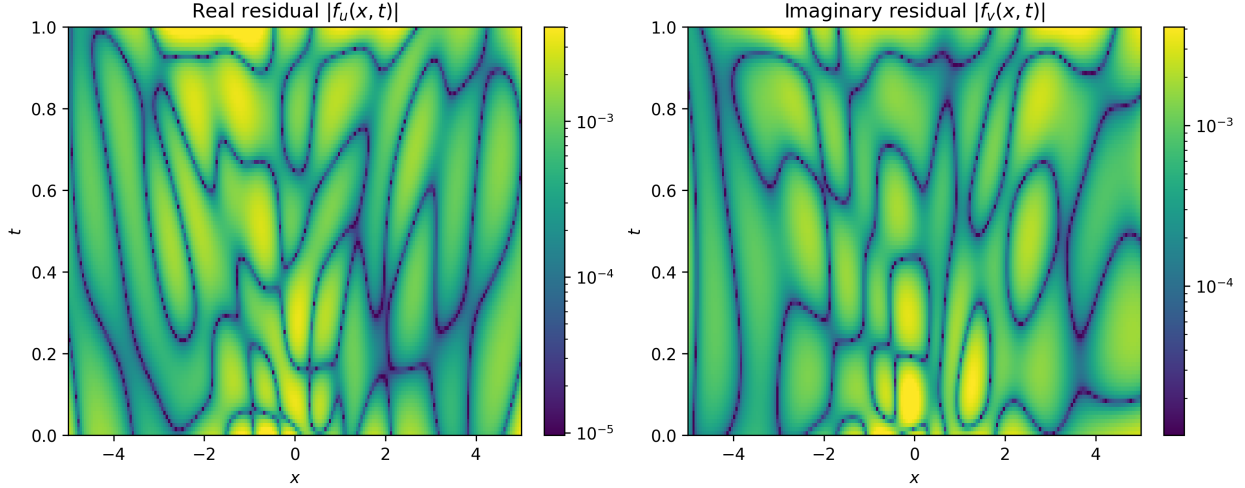


Figure 6: Absolute values of the real and imaginary Schrödinger equation residuals over the computational domain. A logarithmic colour scale is used to display the residual magnitudes.

10^{-6} . The numerical agreement between the residual statistics and the loss history confirms that the two diagnostics are internally consistent.

The maximum values were approximately seven times greater than the corresponding RMS residuals, showing that the remaining PDE mismatch was concentrated in relatively small regions rather than spread uniformly across the domain. No extensive regions of persistently large residual were observed, supporting the conclusion that the PINN satisfied the governing physics globally. The residual statistics are summarized in Table 3.

Table 3: Statistical measures of the real and imaginary PDE residuals.

Statistic	f_u	f_v
Mean absolute residual	7.5442×10^{-4}	7.5263×10^{-4}
RMS residual	1.0379×10^{-3}	1.0209×10^{-3}
Maximum absolute residual	7.4639×10^{-3}	6.7442×10^{-3}

An additional assessment of physical consistency was performed by calculating the integrated probability density:

$$N(t) = \int_{-5}^5 |\psi(x, t)|^2 dx. \quad (38)$$

For Schrödinger evolution under a real potential, this quantity should remain constant in time. The finite-difference reference solution preserved the norm almost exactly, producing a horizontal curve over the full temporal interval. This behaviour is expected because the matrix-exponential time evolution used by the reference solver preserves the discrete wave-function norm to numerical precision.

The PINN result was also nearly constant but displayed small oscillations around the reference level. These fluctuations arise because the neural network satisfies the governing equation and constraints approximately rather than exactly. Importantly, no systematic increasing or decreasing

trend was observed, indicating that the model did not experience substantial accumulated norm drift.

The initial norm was approximately 1.2533, rather than unity. Therefore, the plotted quantity should be described as the wave-function norm rather than total probability. This value indicates that the initial wave function used in the experiment was not normalized to one. Although the lack of normalization does not affect the comparison between the PINN and FDM solutions, normalization would be preferable for a standard quantum-mechanical interpretation.

The temporal evolution of the integrated density is displayed in Figure 7.

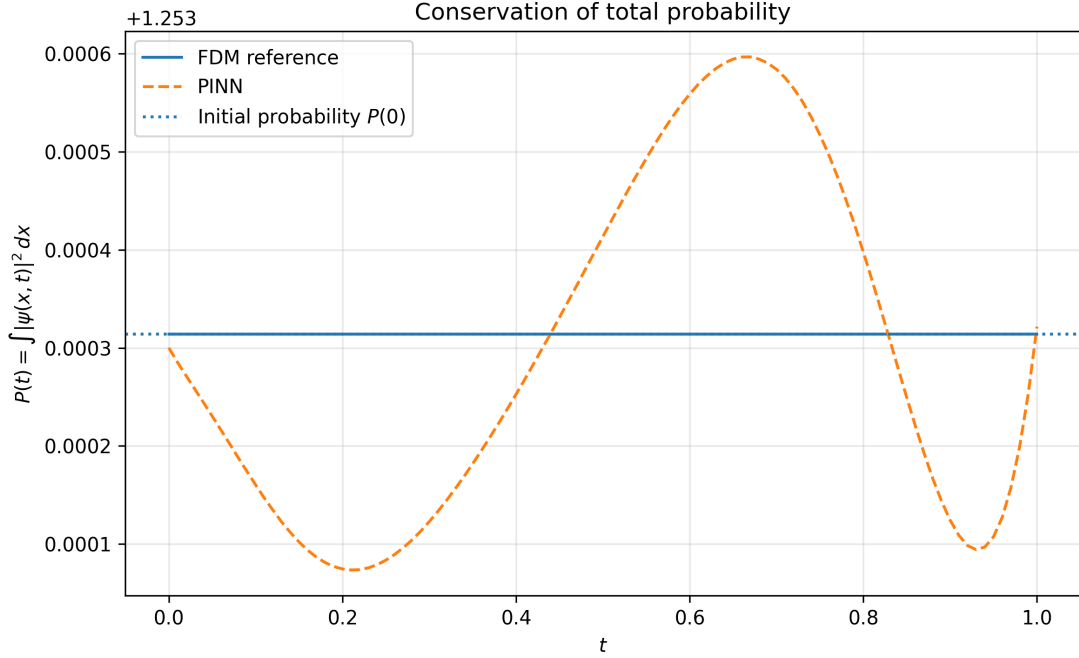


Figure 7: Temporal evolution of the wave-function norm obtained from the PINN and FDM solutions.

The maximum absolute PINN norm deviation was

$$2.9667 \times 10^{-4}, \quad (39)$$

corresponding to a maximum relative deviation of

$$2.3671 \times 10^{-4} \approx 0.0237\%. \quad (40)$$

The finite-difference norm deviation was approximately 7.4303×10^{-13} in relative terms, which is effectively at machine precision. The maximum difference between the PINN and FDM norms was 2.8269×10^{-4} . These results indicate that the PINN preserved the norm with high accuracy despite not imposing exact norm conservation as a separate training constraint. The corresponding conservation measures are provided in Table 4.

The combined numerical evidence demonstrates that the proposed PINN accurately approximated the solution of the time-dependent Schrödinger equation over the considered domain. Stable convergence was achieved through the hybrid Adam and L-BFGS optimization procedure, resulting in

Table 4: Wave-function norm conservation for the PINN and FDM solutions.

Norm measure	Value
Initial PINN norm	1.25330016
Minimum PINN norm	1.25307344
Maximum PINN norm	1.25359683
Maximum absolute PINN norm deviation	2.9667×10^{-4}
Maximum relative PINN norm deviation	2.3671×10^{-4}
Initial FDM norm	1.25331414
Maximum absolute FDM norm deviation	9.3126×10^{-13}
Maximum relative FDM norm deviation	7.4303×10^{-13}
Maximum PINN–FDM norm difference	2.8269×10^{-4}

a final loss of 2.4256×10^{-6} . The boundary and initial conditions were satisfied accurately, while the PDE residual formed the largest remaining contribution to the loss.

Strong agreement was obtained between the PINN and finite-difference solutions at both the field and profile levels. The relative L_2 error of the complete wave function was approximately 0.155%, while the corresponding density error was approximately 0.097%. The maximum pointwise density error remained limited to 1.4054×10^{-3} , and the much smaller MAE and RMSE values showed that this maximum discrepancy occurred only in localized regions.

The residual analysis confirmed that the small prediction error was accompanied by satisfactory enforcement of the governing equation. Both real and imaginary residuals had RMS magnitudes of approximately 10^{-3} , which was consistent with their squared loss contributions of approximately 10^{-6} . The close similarity between the residual statistics of the two equations further indicated that neither component of the coupled system was substantially less well represented.

Physical consistency was also maintained throughout the simulation. The PINN preserved the wave-function norm with a maximum relative deviation of only 0.0237%. Although this level of conservation was less exact than that of the finite-difference reference method, it remained sufficiently small to confirm that the predicted evolution did not suffer from significant artificial gain or loss of norm.

Several limitations should nevertheless be acknowledged. The present model was evaluated for one potential, one temporal interval, and one network configuration. In addition, reference-data constraints were included during training, meaning that the proposed model represents a data-assisted PINN rather than a completely data-free formulation. Further experiments should therefore examine the performance of the model without reference data, with different numbers of collocation points, and with alternative network depths and widths. Longer temporal intervals and more complicated potentials would also provide a stronger assessment of generalization.

The initial wave function should be normalized in future experiments so that the conserved integral equals one and can be interpreted directly as total probability. An ablation study comparing Adam-only training with the hybrid Adam–L-BFGS procedure would additionally clarify the contribution of the second optimization stage. Despite these limitations, the current results provide strong evidence that the proposed PINN formulation can accurately reproduce the dynamics of the investigated Schrödinger system while satisfying the governing equation and preserving its principal integral property.

4 Conclusion

In this study, a physics-informed neural network was developed to solve the one-dimensional time-dependent Schrödinger equation with a harmonic potential, and its predictions were compared with a finite-difference reference solution. The numerical results showed a close agreement between the two approaches over the entire space-time domain. The PINN reproduced the evolution of the wave function and probability density with small relative errors, while the low PDE residuals confirmed that the governing equation was satisfied accurately. The model also preserved the wave-function norm with only a small deviation, although the finite-difference method achieved conservation to near machine precision.

Overall, the results demonstrate that the proposed data-assisted PINN is a reliable approach for approximating the time-dependent Schrödinger equation. While the finite-difference method remains highly accurate for the structured one-dimensional problem considered here, the PINN provides a continuous and flexible approximation that can be evaluated at arbitrary points without constructing a new spatial grid. The ideas presented in this work may therefore support the further development of physics-informed models for more complex quantum systems, including problems with time-dependent potentials, longer simulation intervals, and higher-dimensional domains. Future studies should also examine purely physics-informed formulations without reference data and investigate the influence of network architecture, collocation-point distribution, and optimization strategy on the accuracy and stability of the solution.

References

- [1] Martín Abadi et al. “{TensorFlow}: a system for {Large-Scale} machine learning”. In: *12th USENIX symposium on operating systems design and implementation (OSDI 16)*. 2016, pp. 265–283.
- [2] Salvatore Cuomo et al. “Scientific Machine Learning Through Physics-Informed Neural Networks: Where We Are and What’s Next”. In: *Journal of Scientific Computing* 92.3 (Sept. 2022), p. 88. ISSN: 0885-7474, 1573-7691. DOI: 10.1007/s10915-022-01939-z. URL: <https://link.springer.com/10.1007/s10915-022-01939-z> (visited on 02/24/2026).
- [3] M. W. M. G. Dissanayake and N. Phan-Thien. “Neural-network-based Approximations for Solving Partial Differential Equations”. In: *Communications in Numerical Methods in Engineering* 10.3 (Mar. 1994), pp. 195–201. ISSN: 1069-8299, 1099-0887. DOI: 10.1002/cnm.1640100303. URL: <https://onlinelibrary.wiley.com/doi/10.1002/cnm.1640100303> (visited on 07/11/2026).
- [4] Ehsan Haghighat and Ruben Juanes. “SciANN: A Keras/Tensorflow Wrapper for Scientific Computations and Physics-Informed Deep Learning Using Artificial Neural Networks”. In: *Computer Methods in Applied Mechanics and Engineering* 373 (Jan. 2021), p. 113552. ISSN: 00457825. DOI: 10.1016/j.cma.2020.113552. arXiv: 2005.08803 [cs.OH]. URL: <http://arxiv.org/abs/2005.08803> (visited on 07/12/2026).
- [5] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. “Multilayer Feedforward Networks Are Universal Approximators”. In: *Neural Networks* 2.5 (Jan. 1989), pp. 359–366. ISSN: 08936080. DOI: 10.1016/0893-6080(89)90020-8. URL: <https://linkinghub.elsevier.com/retrieve/pii/0893608089900208> (visited on 07/11/2026).

- [6] Nikhil Ketkar. “Introduction to keras”. In: *Deep learning with python: a hands-on introduction*. Springer, 2017, pp. 97–111.
- [7] Risi Kondor and Shubhendu Trivedi. “On the generalization of equivariance and convolution in neural networks to the action of compact groups”. In: *International conference on machine learning*. PMLR, 2018, pp. 2747–2755.
- [8] Manoj Kumar and Neha Yadav. “Multilayer Perceptrons and Radial Basis Function Neural Network Methods for the Solution of Differential Equations: A Survey”. In: *Computers & Mathematics with Applications* 62.10 (Nov. 2011), pp. 3796–3811. ISSN: 08981221. DOI: 10.1016/j.camwa.2011.09.028. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0898122111007966> (visited on 07/11/2026).
- [9] I.E. Lagaris, A. Likas, and D.I. Fotiadis. “Artificial Neural Networks for Solving Ordinary and Partial Differential Equations”. In: *IEEE Transactions on Neural Networks* 9.5 (Sept. 1998), pp. 987–1000. ISSN: 10459227. DOI: 10.1109/72.712178. URL: <http://ieeexplore.ieee.org/document/712178/> (visited on 07/11/2026).
- [10] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. “Deep Learning”. In: *Nature* 521.7553 (May 28, 2015), pp. 436–444. ISSN: 0028-0836, 1476-4687. DOI: 10.1038/nature14539. URL: <https://www.nature.com/articles/nature14539> (visited on 07/11/2026).
- [11] Dimitris C. Psychogios and Lyle H. Ungar. “A Hybrid Neural Network-first Principles Approach to Process Modeling”. In: *AIChE Journal* 38.10 (Oct. 1992), pp. 1499–1511. ISSN: 0001-1541, 1547-5905. DOI: 10.1002/aic.690381003. URL: <https://aiche.onlinelibrary.wiley.com/doi/10.1002/aic.690381003> (visited on 07/11/2026).
- [12] M. Raissi, P. Perdikaris, and G.E. Karniadakis. “Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear Partial Differential Equations”. In: *Journal of Computational Physics* 378 (Feb. 2019), pp. 686–707. ISSN: 00219991. DOI: 10.1016/j.jcp.2018.10.045. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0021999118307125> (visited on 06/13/2026).
- [13] Svetislav Savović, Miloš Ivanović, and Rui Min. “A Comparative Study of the Explicit Finite Difference Method and Physics-Informed Neural Networks for Solving the Burgers’ Equation”. In: *Axioms* 12.10 (Oct. 18, 2023), p. 982. ISSN: 2075-1680. DOI: 10.3390/axioms12100982. URL: <https://www.mdpi.com/2075-1680/12/10/982> (visited on 07/13/2026).
- [14] Ramy Tanios. “Physics Informed Neural Networks in Computational Finance: High Dimensional Forward & Inverse Option Pricing”. In: (June 27, 2021), 59 p. DOI: 10.3929/ETHZ-B-000491555. HDL: 20.500.11850/491555. URL: <http://hdl.handle.net/20.500.11850/491555> (visited on 07/13/2026).
- [15] Pantelis R. Vlachas et al. “Data-Driven Forecasting of High-Dimensional Chaotic Systems with Long Short-Term Memory Networks”. In: *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences* 474.2213 (May 2018), p. 20170844. ISSN: 1364-5021, 1471-2946. DOI: 10.1098/rspa.2017.0844. URL: <http://royalsocietypublishing.org/rspa/article/56754> (visited on 07/11/2026).

- [16] Li Wang and Zhenya Yan. “Data-Driven Rogue Waves and Parameter Discovery in the Defocusing Nonlinear Schrödinger Equation with a Potential Using the PINN Deep Learning”. In: *Physics Letters A* 404 (July 2021), p. 127408. ISSN: 03759601. DOI: 10.1016/j.physleta.2021.127408. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0375960121002723> (visited on 07/08/2026).
- [17] Hui Xu, Wei Zhang, and Yong Wang. “Explore Missing Flow Dynamics by Physics-Informed Deep Learning: The Parameterized Governing Systems”. In: *Physics of Fluids* 33.9 (Sept. 1, 2021), p. 095116. ISSN: 1070-6631, 1089-7666. DOI: 10.1063/5.0062377. URL: <https://pubs.aip.org/pof/article/33/9/095116/1064746/Explore-missing-flow-dynamics-by-physics-informed> (visited on 07/13/2026).
- [18] Qiming Zhu, Zeliang Liu, and Jinhui Yan. *Machine Learning for Metal Additive Manufacturing: Predicting Temperature and Melt Pool Fluid Dynamics Using Physics-Informed Neural Networks*. Sept. 16, 2020. DOI: 10.48550/arXiv.2008.13547. arXiv: 2008.13547 [cs.CE]. URL: <http://arxiv.org/abs/2008.13547> (visited on 07/13/2026). Pre-published.